

Python Data Cleaning Project Documentation

Dataset Selection

The sample analysis questions compare crime rates against socio-economic conditions, I chose police forces that give a wide spread of different contexts. The goal was simply to make the comparisons more interesting and diverse.

Metropolitan Police (London)

- Huge population, highest crime volume
- Fully urban, megacity dynamics
- Strong anchor for rate comparisons
- Long, stable data collection history

West Midlands Police

- Second-largest force by population
- Much less affluent than London
- Useful contrast between two major urban forces

Surrey Police

- Wealthy, low-density, semi-rural
- Helps widen the socio-economic range

Northumbria Police

- Historically more deprived
- Adds northern geographic coverage and variation

Enrichment Datasets

Chosen mainly because Q6 asks directly about population and socio-economic context.

1. ONS mid-year population estimates
2. IMD 2025

3. LSOA boundaries dataset

IMD includes a Crime domain, so using it to explain crime patterns is slightly circular — but unavoidable if we want deprivation context.

Dataset Sources

1. data.police.uk/data — main police force data
2. [ONS mid-year population estimates](#)
3. [IMD 2025](#)
4. [LSOA boundaries](#)
5. [LAD to CSP to PFA lookup](#)

Key Decisions & Assumptions

- Used 3 years of crime data — long enough for seasonal patterns to repeat, but not so long that the dataset becomes unwieldy. (Was also running out of storage space)
- Population applied as a flat annual estimate rather than interpolated monthly. Month-to-month changes are tiny, and interpolation would add unnecessary assumptions. Also, the required 2025/2026 estimates aren't published yet.

Ingestion Layer

During ingestion, each raw file was immediately reduced to **LSOA × month × crime type counts**. This keeps memory usage low while preserving enough detail for cleaning and enrichment later. I was intending to produce two separate grains, LSOA x crime type x month and Police Force x crimetype x Month; the second is fully derivable from the first so there was no need to keep the entire dataset pre aggregation.

Duplicate Handling

Duplicates were checked *before* aggregation, because once you collapse to LSOA × month × crime type, duplicates disappear.

- Found **384,822 exact duplicate rows** (~7.4% of raw data), mostly Anti-social behaviour.
- ASB has intentionally blank Crime IDs and sparse detail, so different incidents can look identical.
- Lat/long are snapped to a list of default locations, so crimes in similar locations often have the exact same lat/long. More detail in notebook.

Final approach:

Final Approach

- De-duplicate only rows with a **populated Crime ID**.
- Leave null-ID rows untouched to avoid undercounting crime types. I made this final decision to remove only those crimes which I could be 100% sure were duplicates. Again, my full thought process is explained in the notebook.

Force Column

Used **Falls within** instead of **Reported by**, since it reflects jurisdiction. Both columns were identical across all files.

Columns Dropped at Ingestion

Dropped columns that don't matter once we aggregate: `Crime ID`, `Falls within`, `Longitude`, `Latitude`, `Location`, `Last outcome category`, `Context`.

Validation

Confirmed raw row totals matched the aggregated crime counts.

Enrichment Ingestion

- IMD: trivial.
- Population: needed a specific sheet; included age breakdowns, which were outside of the scope of this analysis.
- LSOA count mismatch traced to Wales — population covers England & Wales, IMD covers England only.
- LSOA boundaries ingested via geopandas; counts matched population data.

Cleaning & Validation Layer

Ran a full data quality assessment: shapes, dtypes, nulls, unique counts, date ranges, category consistency, duplicates, and aggregation checks.

Key Findings

- No nulls in crime data.
- 36 unique year-month combinations, 14 crime types — clean and consistent.
- Found **13,996 unique LSOA codes**, which was higher than expected.

Jurisdiction Investigation

Many crimes appeared outside their reporting force's jurisdiction. Verified this wasn't a pipeline bug by checking raw files.

Steps taken:

- Used LAD→PFA lookup to check LSOA codes.
- Built an LSOA→PFA mapping and joined it onto the crime data.
- Added a `jurisdiction_mismatch` flag.
- Relabelled mismatched LSOAs as **Out of bounds** — kept them because they're still valid at non-geographic grains.
- Confirmed no duplicates at the LSOA × crime type × month grain.

Enrichment Joins

- Joined population and IMD via left joins.
- Dropped duplicate LSOA columns and temporary working columns.
- Nulls introduced for "Out of bounds" rows left intentionally.

Temporal Mismatch

Crime data is monthly; population and IMD are static snapshots. Both applied as constants across the whole window — unavoidable given public data availability.

Feature Engineering & Transformation

Derived time attributes (Year, Month Number, Month Name). Joined population and IMD directly at LSOA level.

Chose **IMD decile** over raw rank because of interpretability.

Boundary geometry wasn't merged into the main file on Miguel's advice.

Aggregation for Reporting

Two reporting grains were produced:

1) LSOA × Month × Crime Type

Fully derivable from the enriched dataset — no extra joins needed.

2) Force × Month × Crime Type

2) Force × Month × Crime Type

Even though Power BI *could* aggregate this, I generated it explicitly in Python to avoid mistakes.

- Crime counts summed across LSOAs.
- Population summed once per unique LSOA.
- IMD averaged.
- Used a deduplicated LSOA-level lookup to avoid overcounting.

Metrics Included

- `crime_count`
- `crime_rate_per_1000`
- `Population`
- `IMD_Decile` / `Mean IMD Decile`
- `crime_category_share` — shows crime mix proportions, not just raw counts.

Validation

- Population and IMD showed exactly one value per force.
- Force-level population matched manual sums.
- Category shares summed to ~1 per group.
- Total crime counts reconciled perfectly between both grains.

Export Layer

Exported both reporting datasets to CSV:

- `force_grain.csv`
- `lsoa_grain.csv`